Clumpy Medium Extension of LaRT: Algorithm Description (v2.10)

Contents

1. Overview

The clumpy medium mode adds a population of N non-overlapping spherical clumps inside a spherical domain of radius R. Each clump is a sphere of radius r_c with uniform Lyman-α opacity; the inter-clump medium is vacuum.

In **LaRT_v2.10** the mode is activated by setting

```
par%grid_type = 'clump'
```

in the input namelist. The legacy flag

```
par%use_clump_medium = .true.
```

is still accepted and is mapped to `par%grid_type = 'clump'` during input parsing (backward compatibility with v2.00 input files).

New source files introduced for this mode:

File	Role
`clump_mod.f90`	Clump population: init, RSA placement, CSR grid, raytrace helpers
`raytrace_clump.f90`	Raytrace through the clump medium (DDA + sphere intersection)
`grid_mod_clump.f90`	Grid setup: creates Cartesian shell grid, then calls `init_clumps`

The OOP dispatch is handled by `grid_class.f90` through the concrete type `grid_clump_type` (see §2.).

2. OOP Architecture (v2.10)

2.1 Abstract base type

`grid_class.f90` defines an abstract base type:

```
type, abstract :: grid_base_type
contains
  procedure(raytrace_tau_if),  deferred :: raytrace_to_tau
  procedure(raytrace_edge_if), deferred :: raytrace_to_edge
  procedure(scatter_if),       deferred :: scattering
  procedure(output_if),        deferred :: output_reduce
  procedure(output_if),        deferred :: output_normalize
  procedure(write_if),         deferred :: write_output
  procedure(peel_simple_if),   deferred :: peeling_direct
  ...
  procedure(grid_init_if),     deferred :: grid_init
  procedure(grid_destroy_if),  deferred :: grid_destroy
end type grid_base_type
```

2.2 Concrete clump type

`grid_clump_type` extends `grid_base_type`:

```
type, extends(grid_base_type) :: grid_clump_type
  type(grid_type) :: g     ! Cartesian shell grid (uniform clump parameters)
contains
  procedure :: raytrace_to_tau    => clump_raytrace_to_tau
  procedure :: raytrace_to_edge   => clump_raytrace_to_edge
  procedure :: add_escaped_to_jout=> clump_add_escaped_to_jout
  procedure :: generate_photon    => clump_generate_photon
  procedure :: scattering         => clump_scattering
```

```
  procedure :: output_reduce      => clump_output_reduce
  procedure :: output_normalize   => clump_output_normalize
  procedure :: write_output       => clump_write_output
  procedure :: observer_create    => clump_observer_create
  procedure :: observer_destroy   => clump_observer_destroy
  procedure :: make_sightline_tau => clump_make_sightline_tau
  procedure :: grid_init          => clump_grid_init
  procedure :: grid_destroy       => clump_grid_destroy
end type grid_clump_type
```

All procedures delegate to the same underlying subroutines used in v2.00 (`raytrace_clump_mod`, `scattering_car`, etc.).

2.3 Runtime selection in `main.f90`

Grid selection is performed once at startup:

```
class(grid_base_type), allocatable :: grid

select case (trim(par%grid_type))
case ('clump')
  allocate(grid_clump_type :: grid)
case ('amr')
  allocate(grid_amr_type :: grid)
case default   ! 'car'
  allocate(grid_car_type :: grid)
end select

call grid%grid_init()
...
call grid%output_reduce()
call grid%output_normalize()
call grid%write_output(trim(par%out_file))
call grid%grid_destroy()
```

Scatter, peel-off, and observer routines are invoked identically for all grid types through the polymorphic `grid` variable.

2.4 Comparison with v2.00

In v2.00 the same dispatch was performed via module-level procedure pointers declared in `define.f90` and assigned in `setup_procedure()`:

```
! v2.00 approach (procedure pointers in define.f90)
procedure(raytrace_tau_if), pointer :: raytrace_to_tau => null()
```

```
...
! setup.f90: setup_procedure()
if (par%use_clump_medium) then
  raytrace_to_tau => raytrace_to_tau_clump
  ...
end if
```

The v2.10 OOP approach is semantically equivalent but eliminates the global mutable state and makes the dispatch relationship explicit in the type system. The underlying physics routines (`clump_mod.f90`, `raytrace_clump.f90`, `grid_mod_clump.f90`) are *identical* between v2.00 and v2.10.

3. Physical Setup

3.1 Geometry

The outer domain is a sphere of radius $R = $ `par%rmax` (code units). N spherical clumps of radius $r_c = $ `par%clump_radius` are placed at random inside this sphere (non-overlapping).
A Cartesian grid of $N_x \times N_y \times N_z$ cells is created with the same bounding box $[-R, R]^3$. Grid arrays (`rhokap`, `Dfreq`, `voigt_a`, `vf`) are set to uniform values:

- `rhokap` $= 0$ everywhere (opacity lives only inside clumps);
- `Dfreq` $= \Delta\nu_D^{cl}$ (clump Doppler frequency);
- `voigt_a` $= a_V^{cl}$ (clump Voigt parameter);
- `vfx` $=$ `vfy` $=$ `vfz` $= 0$.

Peel-off and scattering routines read these uniform grid values, so no special-casing is needed inside those modules.

3.2 Clump Count

The user specifies one of three equivalent parameters; the code derives N from whichever is set:

From $f_{\rm co3}$ (covering factor): The mean number of clumps intercepted by a radial ray from the center to the sphere surface is

$$f_{\rm co3} = N\,\frac{\pi r_c^2}{\pi R^2}\cdot\frac{R}{\frac{4}{3}\pi R^3/(4\pi R^2)} = \frac{3}{4}N\left(\frac{r_c}{R}\right)^2, \tag{1}$$

hence

$$N = \frac{4}{3}\,f_{\rm co3}\left(\frac{R}{r_c}\right)^2. \tag{2}$$

Input parameter: `par%clump_f_cov`.

From $f_{3\mathrm{ol}}$ (volume filling factor):

$$f_{3\mathrm{ol}} = N\left(\frac{r_c}{R}\right)^3, \qquad N = f_{3\mathrm{ol}}\left(\frac{R}{r_c}\right)^3. \tag{3}$$

Input parameter: `par%clump_f_vol`.

Direct specification: `par%clump_N_clumps`.

3.3 Clump Physical Parameters

The clump temperature T_c defaults to `par%temperature` unless `par%clump_temperature` > 0. Derived quantities:

$$v_{\mathrm{th}} = v_{\mathrm{th},1}\sqrt{T_c}, \quad v_{\mathrm{th},1} = \sqrt{2k_B/m_H} \approx 12.84\,\mathrm{km\,s^{-1}}/\sqrt{10^4\,\mathrm{K}}, \tag{4}$$

$$b_D^{cl} = \sqrt{v_{\mathrm{th}}^2 + b_{\mathrm{turb}}^2}, \quad b_{\mathrm{turb}} = \texttt{par\%bturb}, \tag{5}$$

$$\Delta\nu_D^{cl} = b_D^{cl}/(\lambda_0 \times \mathrm{km2um}), \tag{6}$$

$$a_V^{cl} = \frac{A_{21}}{4\pi\,\Delta\nu_D^{cl}}, \tag{7}$$

where λ_0 is the Lyman-α rest wavelength and b_{turb} = `par%bturb` is the (spatially uniform) turbulent velocity added in quadrature to the clump thermal width (`par%bturb` ≤ 0 recovers the pure thermal width).

Three mutually exclusive parameters specify the opacity of each individual clump (the first positive value in the priority order wins):

1. `par%clump_tau0` — line-center optical depth (dimensionless): The line-center τ of one clump from its center to its surface.

$$\kappa_c = \frac{\tau_0}{\phi(0, a_V^{cl})\, r_c}, \tag{8}$$

where $\phi(x, a)$ is the Voigt profile at dimensionless frequency x.

2. `par%clump_NHI` — clump HI column density [cm^{-2}]: The HI column density of a *single clump* from its center to its surface. This must be clearly distinguished from the *total system* column density (`par%N_HImax`), which is the average HI column density along a radial sightline through the entire clumpy sphere (center to outer boundary, averaged over all sightlines through the random clump population).

Given $N_{\mathrm{HI}}^{cl} = n_H\, r_c\, d_{\mathrm{cm}}$, the opacity follows as

$$\kappa_c = \frac{N_{\mathrm{HI}}^{cl}\,\sigma_0}{r_c}, \tag{9}$$

where $\sigma_0 \equiv$ `line%cross0`$/\Delta\nu_D^{cl}$ [cm^2] is the line-center cross-section. (`line%cross0` $= \pi e^2 f/(m_e c\sqrt{\pi})$ has units [cm^2 Hz]; dividing by $\Delta\nu_D^{cl}$ [Hz] gives the physical cross-section [cm^2].) This is equivalent to $\tau_0 = N_{\mathrm{HI}}^{cl}\,\sigma_0\,\phi(0, a_V^{cl})$. Unlike `clump_nH`, this option does *not* require `par%distance_unit`.

3. `par%clump_nH` — hydrogen number density [cm^{-3}]: $\kappa_c = n_H\,\sigma_0\,d_{\mathrm{cm}}$, where $\sigma_0 \equiv$ `line%cross0`$/\Delta\nu_D^{cl}$ [cm^2] (requires `par%distance_unit`).

4. System-level fallback (`par%taumax`, `par%tauhomo`, `par%N_HImax`, or `par%N_HIhomo`): If none of the clump inputs above are given, LaRT can fall back to any of four system-level targets (priority order, highest first): `par%taumax`, `par%tauhomo`, `par%N_gasmax` (= `par%N_HImax` for an HI line), and `par%N_gashomo` (= `par%N_HIhomo`). The path taken depends on whether the clump population is generated internally or loaded from a file:

Generated case (no `clump_input_file`*).* For `par%taumax` or `par%N_HImax`, `init_clumps` back-solves the peak clump opacity so the radial-sightline target is realised. For uniform clumps this inverts the closed forms

$$\tau_{\rm max} \;=\; \tfrac{4}{3}\, f_{\rm co3}\, \tau_{c,0}, \qquad N_{\rm HI,max} \;=\; \tfrac{4}{3}\, f_{\rm co3}\, N_{\rm HI}^{cl}; \tag{10}$$

for profile mode it inverts the radial quadrature

$$\tau_{\rm max} \;=\; \frac{4\pi}{3} A_{\rm norm}\, r_{c,0}^3\, \kappa_{c,0}\, \phi(0, a_V^{cl}) \int_{r_{\rm min}}^{R} S_n(r)\, S_r(r)^3\, S_d(r)\, dr, \tag{11}$$

where S_n, S_r, S_d are the number, radius, and density shape factors, and $A_{\rm norm}$ is fixed by the population spec (`clump_N_clumps`, `clump_f_vol`, or `clump_f_cov`) before opacity is computed. An identical expression with $\phi(0, a)$ replaced by $\Delta\nu_D^{cl}/\sigma_0$ governs `N_HImax`. `par%tauhomo` / `par%N_gashomo` are not yet handled by the generated-case back-solve.

Loaded case (`par%clump_input_file`*).* Clump opacities are loaded from the file as-is. If any of the four system-level targets is specified, `rescale_loaded_clumps_to_target` computes the realised value of the selected target from the clump arrays via `compute_clump_scalars` (Eqs. 12 and 13 below) and multiplies every `cl_rhokap(i)` by a single factor α = target/realised, preserving the relative density structure of the file. Priority among the four targets is `taumax` > `tauhomo` > `N_gasmax` > `N_gashomo`; only the chosen target is matched exactly, while the other three are subsequently derived consistently from the rescaled distribution. If none of the four targets is set, the loaded `cl_rhokap(i)` are used unchanged.

3.4 Derived Global Optical-Depth Quantities ($\tau_{\rm homo}$, $N_{\rm HI,homo}$, $\tau_{\rm max}$, $N_{\rm HI,max}$)

LaRT carries four scalar parameters that describe the line-center optical depth and HI column density of the medium as a whole:

`par%tauhomo`, `par%N_gashomo` — the line-center optical depth and HI column density of an equivalent *homogeneous* medium that contains the same total HI mass spread uniformly over the sphere volume.

`par%taumax`, `par%N_gasmax` — the line-center optical depth and HI column density along a single radial sightline from the center to the $+z$ surface.

For a homogeneous medium these four numbers are equal up to geometric factors; for a clumpy medium `taumax` along any specific sightline fluctuates strongly (it can be zero if no clumps lie along the LOS, or many times the homogeneous value if several clumps are stacked), while `tauhomo` is a deterministic property of the medium (mass / volume).

Unified definition for each clump. All four scalars are computed from the clump arrays $\{\kappa_i, a_{V,i}, \Delta\nu_{D,i}, r_i, \mathbf{r}_i\}$ by a single routine, `compute_clump_scalars`, regardless of whether the clumps were generated or loaded from a file:

$$\tau_{\rm homo} = \sum_i \frac{\kappa_i\,\phi(0,a_{V,i})\,r_i^3}{R^2 + R\,r_{\rm min} + r_{\rm min}^2}, \tag{12}$$

$$\tau_{\rm max} = \sum_i \frac{\kappa_i\,\phi(0,a_{V,i})\,r_i^3}{3\,\max(d_i^2,\, r_i^2)}, \tag{13}$$

with $d_i = |\mathbf{r}_i|$ (clump-centre distance from the box origin). $N_{\rm HI,homo}$ and $N_{\rm HI,max}$ use the same forms with $\phi(0, a_{V,i})$ replaced by $\Delta\nu_{D,i}/\sigma_0$. Equation (12) is the uniform-smear interpretation (every clump's opacity is redistributed over the shell of volume $V_{\rm shell} = \frac{4}{3}\pi(R^3 - r_{\rm min}^3)$ and the smeared column is traversed radially across thickness $R - r_{\rm min}$). Equation (13) is the small-angle expected radial line-centre τ from the origin: each clump contributes its expected chord depth $\frac{4}{3}\kappa_i\phi(0,a_{V,i})r_i$ weighted by the solid angle $\pi r_i^2/d_i^2$ it subtends at the origin; the cap $\max(d_i^2, r_i^2)$ removes the divergence for a clump that straddles the origin. In the limit of a uniform-isotropic distribution of identical clumps the two definitions coincide. The auto-detection of the simulation frequency range (`par%xfreq_max`/`xfreq_min`) inside `car_setup_freq_grid` uses both quantities:

$$a_{\tau,3} = (a_V \cdot \tau_{\rm homo})^{1/3}, \qquad x_{\rm max} \approx x_{\rm scale}(\tau_{\rm max}) \cdot a_{\tau,3}. \tag{14}$$

Hence even when the user supplies `velocity_min`/`velocity_max` explicitly, having physically meaningful `tauhomo` and `taumax` is important for the FITS header to be informative and for the fall-back auto-range to produce a usable grid.

Derivation in clump mode. `init_clumps()` is called BEFORE `grid_create()` inside `grid_create_clump`, so the realised clump arrays $\{\kappa_i, a_{V,i}, \Delta\nu_{D,i}, r_i, \mathbf{r}_i\}$ are known by the time `compute_clump_scalars` (Eqs. 12, 13) is invoked from `grid_create_clump` to assign `par%tauhomo`, `par%taumax`, `par%N_gashomo`, and `par%N_gasmax`. When clumps populate the radial shell $r_{\rm min} \le r \le R$ (the default `par%rmin` = −999 is treated as $r_{\rm min} = 0$, recovering the full-sphere case), the volume filling factor and the LaRT covering factor are

$$f_{\rm 3ol} = \frac{N_c\, r_c^3}{R^3 - r_{\rm min}^3}, \qquad f_{\rm co3} = \frac{3}{4}\,\frac{N_c\, r_c^2}{R^2 + R\,r_{\rm min} + r_{\rm min}^2}, \tag{15}$$

which with $r_{\rm min} = 0$ reduce to $N_c(r_c/R)^3$ and $\frac{3}{4}N_c(r_c/R)^2$.

Uniform clumps (closed-form limit). With all $\kappa_i = \kappa_c$, $a_{V,i} = a_V^{cl}$, $r_i = r_c$, the shell-aware sum over clumps Eq. (12) collapses to

$$\tau_{\rm homo} = \tfrac{4}{3}\, f_{\rm co3}\, \tau_0, \qquad \tau_0 = \kappa_c\, \phi(0, a_V^{cl})\, r_c, \tag{16}$$

identical to the previous closed-form derivation. For an isotropic uniform spatial distribution, the small-angle radial-sightline sum Eq. (13) has the same expectation (using $\langle 1/d_i^2 \rangle = 3/(R^2 + R\,r_{\rm min} + r_{\rm min}^2)$), so $\tau_{\rm homo}$ and $\tau_{\rm max}$ coincide to within finite-N sampling noise. The HI columns follow by replacing τ_0 with the clump column $N_{\rm HI}^{cl}$:

$$N_{\rm HI,homo} = N_{\rm HI,max}^{\rm (expected)} = \tfrac{4}{3}\, f_{\rm co3}\, N_{\rm HI}^{cl}, \qquad N_{\rm HI}^{cl} = \frac{\kappa_c\, \Delta\nu_D^{cl}}{\texttt{line\%cross0}}\, r_c. \tag{17}$$

For the canonical test case (`clump_NHI` = 7.5e17 cm^{-2}, $f_{\rm co3} = 1$, $T = 10^4$ K) this gives $N_{\rm HI,homo} = 1.0 \times 10^{18}$ cm^{-2}, $\tau_{\rm homo} = 5.9 \times 10^4$, and $a_{\tau,3} = 3.0$. When the user supplies `par%N_HImax` (or `par%taumax`) and lets `init_clumps` back-solve κ_c

via the shell formula, the realised $N_{\rm HI,homo}$ matches the input target exactly (modulo `nint` rounding of N_c). Example: `clump_NHI18_fcov1.in` with `par%rmin` = 0.1, $f_{\rm co3} = 1$, `par%N_HImax` = 1.0×10^{18} reports $N_{\rm gashomo} = 1.000 \times 10^{18}$ as expected.

Non-uniform clumps (loaded or profile mode). For a population loaded from `par%clump_input_file` the clump sums in Eqs. (12), (13) are evaluated directly on the actual $\{\kappa_i, r_i, \mathbf{r}_i\}$, so $\tau_{\rm homo}$ and $\tau_{\rm max}$ may differ meaningfully when the spatial distribution is anisotropic or concentrated (e.g. a biconical wind). The reported `par%tauhomo`/`par%taumax` values then reflect the actual realised distribution and are always overwritten by `compute_clump_scalars`; if the user supplied one of the four targets, the rescale step (§3.3) ensures the chosen target is reproduced exactly, with the other three derived consistently.

3.5 Clump Velocities

Each clump's velocity is the sum of two independent components:

$$\mathbf{3}_i = \mathbf{3}_{\rm sys}(\mathbf{r}_i) + \mathbf{3}_{\rm rand,i}, \tag{18}$$

where $\mathbf{3}_{\rm sys}$ is a position-dependent systematic flow (set by `par%velocity_type`) and $\mathbf{3}_{\rm rand}$ is an independent Gaussian random component (set by `par%clump_sigma_v`). Either component can be zero.

Random component: If `par%clump_sigma_v` > 0, each clump is assigned $\mathbf{3}_{{\rm rand},i} \sim \mathcal{N}(0, \sigma_v^2)$ per component (drawn in `generate_clumps`).

Systematic component (`velocity_type`): After RSA placement, `assign_clump_velocities_from_type` evaluates the flow velocity at each clump center and *adds* it to the existing (possibly non-zero) velocity. Supported types:

`velocity_type`	Formula	Parameters
`'hubble'`	$\mathbf{3} = V_{\rm exp}\,\mathbf{r}/R$	`Vexp`
`'constant_radial'`	$\mathbf{3} = V_{\rm exp}\,\hat{\mathbf{r}}$	`Vexp`
`'power_law'`	$\mathbf{3} = V_{\rm exp}(r/R)^{\alpha_V}\,\hat{\mathbf{r}}$	`Vexp, velocity_alpha`
`'linear_decelerate'`	$\mathbf{3} = V_{\rm exp}\,\frac{R-r}{R-r_{\rm min}}\,\hat{\mathbf{r}}$	`Vexp`
`'parallel_velocity'`	$\mathbf{3} = (V_x, V_y, V_z)$	`Vx, Vy, Vz`
`'ssh'`	see below	`Vpeak, rpeak, DeltaV`
`'rotating_solid_body'`	$\mathbf{3} = V_{\rm rot}(-y, x, 0)/R$	`Vrot`
`'rotating_galaxy_halo'`	flat rotation about z	`Vrot, rinner`

All velocity magnitudes are in km/s. The `'ssh'` model (Song, Seon & Hwang 2020) gives a linear rise inside $r_{\rm peak}$ and a Hubble-like outer flow:

$$v(r) = \begin{cases} V_{\rm peak}\, r/r_{\rm peak} & r < r_{\rm peak}, \\ V_{\rm peak} + \Delta V\,(r - r_{\rm peak})/(R - r_{\rm peak}) & r \geq r_{\rm peak}. \end{cases} \tag{19}$$

The `'rotating_galaxy_halo'` model uses the cylindrical radius $\rho = \sqrt{x^2+y^2}$ and gives solid-body rotation inside $r_{\rm inner}$ and a flat curve ($v_\phi = V_{\rm rot}$) outside.

4. Clump Placement: Random Sequential Addition (RSA)

4.1 Concept

Random Sequential Addition (RSA) is a simple stochastic algorithm for generating configurations of non-overlapping hard spheres. Candidate positions are drawn at random one at a time; each is tested against all previously accepted clumps and either accepted (no overlap) or discarded irreversibly (overlap detected). Because accepted positions are never moved, RSA requires no iterative relaxation.

The non-overlap condition enforces a minimum center-to-center distance of $2r_c$. RSA reliably converges for volume filling factors $f_{3\mathrm{ol}} \lesssim 35\%$; above this threshold, the fraction of unblocked placement space decreases so steeply that placing the last few clumps may require enormous numbers of attempts. The theoretical 3-D jamming limit for RSA is $f_{3\mathrm{ol}} \approx 38\%$.

4.2 Uniform Sampling Inside the Shell

Candidate positions must be distributed *uniformly* inside the radial shell $r_{\min} \le r \le R$, where $r_{\min} = \max(0, \texttt{par\%rmin})$ and $R = \texttt{par\%rmax}$. The default `par%rmin = -999` is treated as $r_{\min} = 0$ and reproduces the original "filled sphere" behavior; setting $r_{\min} > 0$ carves out an inner cavity in which no clumps are placed.

This is achieved by box-rejection on the spherical shell:

1. Draw x_c, y_c, z_c each uniformly from $[-R, R]$.
2. Compute $d^2 = x_c^2 + y_c^2 + z_c^2$.
3. Accept if $r_{\min}^2 \le d^2 \le R^2$; otherwise draw again.

The acceptance probability is

$$\frac{\pi}{6}\,\frac{R^3 - r_{\min}^3}{R^3}, \tag{20}$$

which reduces to $\pi/6 \approx 52\%$ when $r_{\min} = 0$. Box-rejection avoids trigonometric functions and yields an exactly uniform distribution in 3-D Cartesian space.

4.3 Fully-Inside Constraint

By default the placement requires the *whole* clump to fit inside the shell, not just its center. A clump of radius r_c centered at distance d from the box origin is accepted only when

$$r_{\min} \;\le\; d - r_c \quad \text{and} \quad d + r_c \;\le\; R. \tag{21}$$

This is controlled by the namelist switch

```
par%clump_fully_inside = .true.    ! default
par%rmin               = -999      ! default (no inner cavity)
```

- Uniform-radius case: the placement shell is inset on both sides – from $[r_{\min}, R]$ to $[r_{\min}+r_c, R-r_c]$ – so the box-rejection draw above samples uniformly inside the inset shell. All accepted clumps automatically satisfy Eq. (21); no sample-by-sample rejection is needed.
- Spatially-varying clump radii (`clump_radius_profile` not `'constant'`): the inverse-CDF radial draw is built so that $n_{\rm cl}(r) = 0$ for $r < r_{\min}$, ensuring centers land in the shell automatically. If $d + r_c > R$ or $d - r_c < r_{\min}$, the sample is discarded and a new pair is drawn.
- If $r_{\min} + 2r_{c,\max} > R$ – the shell is too narrow to hold a single clump – the run aborts with an error. Likewise the run aborts if the largest possible clump radius exceeds R (a steep `clump_radius_profile` would yield $r_c \geq R$ near the surface).

Setting `par%clump_fully_inside = .false.` restores the legacy behavior, in which only the clump center is required to lie in $[r_{\min}, R]$. Clumps near either boundary may then protrude past $r = R$ or into the cavity below $r = r_{\min}$; this reproduces results generated before these options were introduced. As an illustration, the `clump_radius_powerlaw` test case (`par%clump_radius` $= 0.01$, $r_0 = 0.5$, $\alpha = -1$, $N = 10^4$, $R = 1$, $r_{\min} = 0$) yields ~ 564 outer-edge protruding clumps with the legacy setting and zero with the new default.

The conversion of `clump_f_vol` or `clump_f_cov` to `N_clumps` uses the shell volume $V_{\rm shell} = \frac{4}{3}\pi(R^3 - r_{\min}^3)$ and the shell sightline factor $R^2 + R\,r_{\min} + r_{\min}^2$, so $f_{\rm 3ol}$ and $f_{\rm co3}$ refer to the shell rather than the full sphere. With $r_{\min} = 0$ these reduce exactly to the original $(R/r_c)^3$ and $(R/r_c)^2$ expressions, preserving prior runs. When `clump_fully_inside` is on, the realized $f_{\rm 3ol}$ and $f_{\rm co3}$ fall slightly below the target whenever r_c is a non-trivial fraction of $R - r_{\min}$, since the available placement volume shrinks from $\frac{4}{3}\pi(R^3 - r_{\min}^3)$ to $\frac{4}{3}\pi\left[(R-r_c)^3 - (r_{\min}+r_c)^3\right]$. Specify a slightly larger $f_{\rm 3ol}/f_{\rm co3}$ if exact matching is required.

4.4 Placement Algorithm

```
do while (icl < N_clumps)

  !--- Step 1: draw candidate inside the shell [rmin, R]
  !            (with R, rmin inset by r_c when clump_fully_inside)
  do
    xc = (2.0*rand() - 1.0) * R_outer
    yc = (2.0*rand() - 1.0) * R_outer
    zc = (2.0*rand() - 1.0) * R_outer
    d2 = xc**2 + yc**2 + zc**2
    if (R_inner**2 <= d2 .and. d2 <= R_outer**2) exit
  enddo

  !--- Step 2: locate candidate in hash-grid cell (ig, jg, kg)
  ig = int((xc - xmin) / cell_size)
  jg = int((yc - ymin) / cell_size)
```

```
  kg = int((zc - zmin) / cell_size)

  !--- Step 3: check 27 neighboring cells for overlap
  overlap = .false.
  outer: do kg2 = kg-1, kg+1
         do jg2 = jg-1, jg+1
         do ig2 = ig-1, ig+1
    if (out of bounds) cycle
    jnb = head(ig2, jg2, kg2)       ! first clump in this hash cell
    do while (jnb > 0)
      if (dist2(xc,yc,zc, cl_x(jnb),cl_y(jnb),cl_z(jnb)) < (2*rc)**2) then
        overlap = .true.; exit outer ! reject immediately on first overlap
      endif
      jnb = nxt(jnb)                ! follow linked list to next clump
    enddo
  enddo; enddo; enddo outer

  if (overlap) cycle                ! reject: draw new candidate

  !--- Step 4: accept -- record position and random velocity
  icl = icl + 1
  cl_x(icl) = xc;  cl_y(icl) = yc;  cl_z(icl) = zc
  if (sigma_v > 0.0) then
    cl_vx(icl) = sigma_v * gauss_random()  ! one independent draw per component
    cl_vy(icl) = sigma_v * gauss_random()
    cl_vz(icl) = sigma_v * gauss_random()
  endif

  !--- Step 5: prepend clump to the linked list for cell (ig, jg, kg)
  nxt(icl)         = head(ig, jg, kg)
  head(ig, jg, kg) = icl

enddo
```

4.5 Linked-List Hash Grid and the 27-Cell Rule

Naively, checking candidate n against all $n-1$ accepted clumps costs $O(n)$, giving $O(N^2)$ total work. The linked-list hash grid reduces each overlap check to $O(1)$ expected cost by spatially indexing accepted clumps.

The domain $[-R,R]^3$ is partitioned into N_g^3 uniform cells with

$$N_g = \min\big(512,\ \lfloor N^{1/3} \rfloor + 1\big), \qquad \ell_{\mathrm{RSA}} = \frac{2R}{N_g}. \tag{22}$$

Two integer arrays encode the linked lists: `head(i,j,k)` is the index of the first clump in cell (i,j,k) (0 if empty), and `nxt(icl)` is the next clump in the same cell list (0 = end of list). When a new clump is accepted (Step 5 above), it is prepended to the head of its cell's list in $O(1)$.

Why 27 cells are sufficient. Two clumps overlap iff their center separation is $< 2r_c$. If the candidate lies in cell (i,j,k), any overlapping clump must satisfy $|x_{\rm cl} - x_c| < 2r_c$. Since $\ell_{\rm RSA} \geq 2r_c$ by construction (eq. 22), the overlapping clump's cell index i' satisfies $i' \in \{i-1, i, i+1\}$, and likewise for j' and k'. Checking all $3^3 = 27$ cells centered on (i,j,k) is therefore both sufficient and complete — no overlapping clump can lie outside this neighborhood.

Cell-size constraint. The 27-cell rule requires $\ell_{\rm RSA} \geq 2r_c$, i.e. $N_g \leq R/r_c$. For the reference case ($R = 1$, $r_c = 0.01$): $R/r_c = 100$, $N_g = \lfloor 66667^{1/3} \rfloor + 1 = 42$, $\ell_{\rm RSA} = 2/42 \approx 0.048 \gg 2r_c = 0.02$. ✓

4.6 Acceptance Rate and Jamming Limit

As more clumps are placed, their exclusion zones (spheres of radius $2r_c$ around each accepted clump) collectively block an increasing fraction of the placement domain. The candidate acceptance probability $p_{\rm acc}$ decreases monotonically with $f_{\rm 3ol}$ and approaches zero at the jamming limit.

For the reference case ($f_{\rm co3} = 5$, $r_c/R = 0.01$, $N = 66667$, $f_{\rm 3ol} = 0.067$), the overall acceptance rate is $\approx 74\%$: roughly 90 000 candidates are drawn to place 66 667 clumps. The algorithm remains practical for $f_{\rm 3ol} \lesssim 35\%$.

4.7 Multi-node Correctness

In a multi-node MPI run, naively executing RSA on each node independently would produce different clump layouts per node (each node has its own random-number state), corrupting photon transport across node boundaries.

Clump positions are therefore generated *only on global rank 0* (`p_rank==0`). The resulting `cl_x/y/z` and `cl_vx/vy/vz` arrays are then broadcast to all other node-head processes via six `MPI_BCAST` calls on `mpar%SAME_HRANK_COMM` (the communicator containing exactly one rank-0 process per node). Ranks within the same node share the arrays through MPI-3 shared memory, requiring no further communication. The CSR build is fully deterministic given the positions, so every node constructs the same acceleration grid without any additional broadcast.

This corrects an earlier bug in which `h_rank==0` (the within-node head rank) triggered independent RSA passes on each node, producing different clump layouts per node.

5. CSR Acceleration Grid

5.1 Motivation

During radiative transfer, a photon in vacuum must find the nearest clump along its current flight direction at every propagation step. Checking all N clumps per step would cost $O(N)$ per query and become prohibitive for large populations ($N \sim 10^4$–10^5). The CSR acceleration grid pre-sorts clumps into spatial cells; the DDA traversal (§6.3) then visits only

the cells the ray crosses, and within each cell tests only the few registered clumps. Together, DDA+CSR achieves $O(1)$ amortised cost per nearest-clump query, independent of N.

5.2 Data Structure: Compressed Sparse Row

Instead of a list-of-lists (which requires many small dynamic allocations), the clump lists for each cell are packed into two flat integer arrays in *Compressed Sparse Row* (CSR) format — the same layout used for sparse matrices:

- `cg_start(0:Ncell)`: `cg_start(icell)` is the 1-based index of the first entry for cell `icell` in `cg_list`. The entries for cell `icell` occupy positions `cg_start(icell) ... cg_start(icell+1)-1`; their count is `cg_start(icell+1) - cg_start(icell)`.
- `cg_list(1:Ntot)`: flat concatenation of the clump-index lists for each cell.

Retrieving the clump list for any cell is an $O(1)$ array slice with no pointer chasing. To illustrate: suppose four cells contain the clumps $\{2,5\}$, $\{\}$, $\{1,3\}$, $\{4\}$ respectively:

$$\texttt{cg\_start} = [1,\ 3,\ 3,\ 5,\ 6],$$

$$\texttt{cg\_list} = [2,\ 5,\ 1,\ 3,\ 4].$$

The clumps in cell 2 are `cg_list(3:4)` $= \{1,3\}$. Both arrays reside in MPI-3 shared memory (one copy per compute node).

5.3 Cell Grid and Domain

The CSR grid reuses the same cell count $M = N_g$ as the RSA hash grid (§4.5), but covers a slightly larger domain $[-R-r_c,\ R+r_c]^3$ with uniform cell size

$$\delta = \frac{2(R+r_c)}{M}. \tag{23}$$

Expanding the domain by r_c in each direction ensures two properties:

1. A clump centered exactly on the outer sphere surface ($|\mathbf{c}| = R$) fits entirely inside the grid.
2. A photon entering from outside the outer sphere starts inside a valid grid cell, so the DDA traversal can begin immediately.

5.4 Clump Registration: AABB Overlap Test

A clump of radius r_c centered at (c_x, c_y, c_z) is registered in every cell whose axis-aligned bounding box (AABB) overlaps the clump sphere. Cell (i,j,k) spans $[x_{\min} + i\,\delta,\ x_{\min} + (i+1)\,\delta]$ along x. The range of cell indices that overlap $[c_x - r_c,\ c_x + r_c]$ is

$$i \in \left[\left\lfloor \frac{c_x - r_c - x_{\min}}{\delta} \right\rfloor,\ \left\lfloor \frac{c_x + r_c - x_{\min}}{\delta} \right\rfloor\right], \tag{24}$$

and likewise for j and k. The clump is registered in every cell in the Cartesian product of these three index ranges. For the reference case ($r_c = 0.01$, $\delta \approx 0.048$, so $r_c/\delta \approx 0.21 < 1$), each clump's index range spans at most 2 values per dimension, so a clump touches at most $2^3 = 8$ cells. The total registration count satisfies $N \le N_{\text{tot}} \le 8N$.

5.5 Two-Pass Construction

The CSR arrays are built in two sequential passes over all N clumps, avoiding any intermediate list storage (the standard technique for CSR sparse-matrix assembly):

Pass 1 (count): Compute the cell ranges (eq. 24) for each clump and increment a counter `cnt(icell)` for every touched cell. Then compute the exclusive prefix sum:

$$\texttt{cg\_start(0)} = 1, \qquad \texttt{cg\_start(icell+1)} = \texttt{cg\_start(icell)} + \texttt{cnt(icell)}.$$

Allocate `cg_list` of length $N_{\rm tot} = \texttt{cg\_start(Ncell)} - 1$ as an MPI-3 shared-memory window.

Pass 2 (fill): Repeat the same cell-range loop. For each touched cell, write the clump index at write pointer `ptr(icell)`, then advance the pointer by one. Initialize `ptr(icell) = cg_start(icell)` before this pass.

```
!--- Pass 1: count registrations per cell
cnt(:) = 0
do icl = 1, N_clumps
  i0 = max(0,   floor((cl_x(icl) - rc - xmin) / delta))
  i1 = min(M-1, floor((cl_x(icl) + rc - xmin) / delta))
  j0 = ...; j1 = ...                    ! analogous for y
  k0 = ...; k1 = ...                    ! analogous for z
  do k = k0,k1;  do j = j0,j1;  do i = i0,i1
    cnt(cell_idx(i,j,k)) = cnt(cell_idx(i,j,k)) + 1
  enddo;  enddo;  enddo
enddo
! exclusive prefix sum -> cg_start
cg_start(0) = 1
do icell = 0, Ncell-1
  cg_start(icell+1) = cg_start(icell) + cnt(icell)
enddo
Ntot = cg_start(Ncell) - 1
! allocate cg_list(Ntot) as MPI-3 shared-memory window

!--- Pass 2: fill cg_list
ptr(0:Ncell-1) = cg_start(0:Ncell-1)   ! one write pointer per cell
do icl = 1, N_clumps
  ... (same i0,i1,j0,j1,k0,k1 as Pass 1)
  do k = k0,k1;  do j = j0,j1;  do i = i0,i1
    ic = cell_idx(i,j,k)
    cg_list(ptr(ic)) = icl
    ptr(ic) = ptr(ic) + 1
  enddo;  enddo;  enddo
```

```
enddo
! invariant: ptr(icell) == cg_start(icell+1) for all cells after Pass 2
```

6. Raytrace Through the Clump Medium

6.1 Overview

Photon propagation is split into two phases:

1. **Advance through vacuum** – use the DDA traversal of the CSR grid (§6.3) to find the next clump hit.
2. **Traverse through a clump** – accumulate optical depth along a straight path through the sphere (§6.4).

Frequency shifts due to individual clump bulk velocities are applied at clump entry and reversed at clump exit.

6.2 Ray–Sphere Intersection

Given a ray $\mathbf{p}(t) = \mathbf{o} + t\hat{\mathbf{k}}$ and a sphere centered at $\mathbf{c}$ with squared radius r_c^2, define $\mathbf{r} = \mathbf{o} - \mathbf{c}$, $b = \mathbf{r} \cdot \hat{\mathbf{k}}$, $\Delta = b^2 - |\mathbf{r}|^2 + r_c^2$. If $\Delta < 0$ there is no intersection. Otherwise

$$t_{\rm entry} = -b - \sqrt{\Delta}, \qquad t_{\rm exit} = -b + \sqrt{\Delta}. \tag{25}$$

A hit is recorded when $t_{\rm exit} > 0$.

6.3 DDA Search for Next Clump

The Amanatides & Woo (1987) digital differential analyser (DDA) traverses the CSR cells along the ray in order of increasing path length. For each cell, all registered clumps are tested with the ray–sphere formula. The traversal stops as soon as the current cell's entry distance exceeds the best hit distance found so far.

```
best_te  = huge;  best_icl = 0
d        = 0     ! path length to current cell entry

do while (.true.)
   if (d > best_te .or. d > t_max) exit
   ! test all clumps in current cell
   do ip = cg_start(icell), cg_start(icell+1)-1
      icl = cg_list(ip)
      if (icl == skip_icl) cycle
      call ray_sphere_isect(..., te, tx2, hit)
      if (hit .and. tx2>0 .and. te<best_te) then
         best_te=te; best_tx2=tx2; best_icl=icl
      endif
   enddo
```

```
15     ! advance to next cell (Amanatides & Woo)
16     if (tx<=ty .and. tx<=tz) then
17        d=tx; ci=ci+si; tx=tx+delx
18     elseif (ty<=tz) then
19        d=ty; cj=cj+sj; ty=ty+dely
20     else
21        d=tz; ck=ck+sk; tz=tz+delz
22     endif
23  enddo
```

6.4 Optical Depth Traversal

`raytrace_to_tau_clump` advances the photon until optical depth $\tau_{\rm in}$ is accumulated or the outer sphere is exited.

Inside a clump (`icell_clump` > 0): The remaining path through the current clump is $s = t_{\rm exit}(\mathbf{p}, \hat{\mathbf{k}}, \text{icl})$. Opacity at the current frequency: $\kappa = \kappa_c\, \phi(x, a_V)$. If $\tau_{\rm rem} \le \kappa s$, scatter at $\mathrm{d}s = \tau_{\rm rem}/\kappa$. Otherwise subtract κs from $\tau_{\rm rem}$, advance to clump exit, shift frequency back to the global frame, and continue in vacuum.

In vacuum (`icell_clump` $= 0$): Call `find_next_clump` (DDA, §6.3) to find the nearest clump. If none is found along the ray, advance to the sphere boundary and exit (`photon%inside = .false.`).

Frequency convention: The photon frequency x (in units of $\Delta\nu_D^{cl}$) is maintained in the local clump rest frame while inside a clump, and in the global frame in vacuum:

$$\text{at clump entry:} \quad x \leftarrow x - u_{\rm los}, \tag{26}$$

$$\text{at clump exit:} \quad x \leftarrow x + u_{\rm los}, \tag{27}$$

where $u_{\rm los} = (\mathbf{3}_i \cdot \hat{\mathbf{k}})/v_{\rm th}$ is the line-of-sight component of the host clump's bulk velocity in units of the clump thermal velocity $v_{\rm th} = \Delta\nu_D^{cl}\,\lambda_0$.

Internal storage of clump bulk velocities: `cl_vx/y/z` are stored as $\mathbf{3}/v_{\rm th}$ (dimensionless), matching the Cartesian/AMR convention for `grid%vfx` (which is also pre-normalized to $v/v_{\rm th}$ in `grid_mod_car.f90`). The clump velocity assignment routines fill the arrays in km/s; `init_clumps` divides by `cl_vtherm` after the `MPI_BCAST` step so that all subsequent raytrace and peel-off code can use `cl_v*` directly in $u_{\rm los}$ without dividing on each call. The user-facing FITS output (Section 8.) multiplies back by `cl_vtherm` to keep the stored `VX/VY/VZ` columns in km/s.

Jout accumulation: When the photon exits the outer sphere (`photon%inside` becomes `.false.`), the photon weight is accumulated into the escape spectrum:

$$J_{\rm out}[i_x] \mathrel{+}= w, \qquad i_x = \left\lfloor \frac{x - x_{\rm min}}{\delta x} \right\rfloor + 1, \tag{28}$$

where x is the photon frequency (global frame, in units of $\Delta\nu_D$) and w is the photon weight. This mirrors the behavior of `raytrace_to_tau_car`, where Jout is accumulated upon grid exit. There are three exit paths (clump-then-sphere exit, already-outside-sphere, and no-clump-found), all handled identically.

6.5 Optical Depth to Edge

`raytrace_to_edge_clump` computes τ from the current position to the sphere boundary without moving the photon (read-only input). The same DDA logic is used; frequency shifts at clump entry/exit are tracked locally. This is called by peel-off routines and by the forced-first-scatter initialization in `run_simulation_mod`.

7. Peel-Off Observations

Peel-off is handled by the standard `peelingoff_rect` routines (`peeling_direct_outside`, `peeling_resonance_nostokes_outside`, etc.). Each of those routines invokes a small mode-aware helper `peel_raytrace_to_edge` (in the same module) which, in clump mode, calls `raytrace_to_edge_clump` – not the Cartesian `raytrace_to_edge_car_dispatch` – so that the peel-off attenuation accumulates the actual clump opacity along the photon-to-observer line of sight (`grid%rhokap` is zero in clump mode and would silently make the Cartesian dispatch return $\tau = 0$, leaving every peeled-off photon unattenuated; this was a silent regression in the v2.10 class-based refactor vs the v2.00 procedure-pointer dispatch and is now fixed). The clump call passes the optional argument `tau_max = tau_huge_clump` ≈ 745 so the integration bails out once $\exp(-\tau)$ underflows in double precision; the sight-line wrappers `raytrace_to_edge_tau_gas_clump` / `raytrace_to_dist_tau_gas_clump` omit `tau_max` and therefore report the FULL integrated tau (sight-line FITS output is unaffected by the cap).

Overlap-aware peel-off attenuation. When `has_overlap = .true.`, `peel_raytrace_to_edge` accumulates optical depth from *all* clumps that intersect the photon-to-observer segment. The non-overlap `raytrace_to_edge_clump` would under-count attenuation when overlapping neighbours are present because it exits as soon as it leaves the current clump. `peel_raytrace_to_edge` therefore branches on `has_overlap` and calls `raytrace_to_edge_clump_overlap` (with `tau_max = tau_huge_clump`) in the overlap case. This variant uses the same event-list walk as the main raytrace (Section 15.) and correctly sums contributions from every clump crossed along the line of sight.
The observer distance defaults to $d = 100\,R$ and the image pixel size is set so that the full sphere subtends the image:

$$\delta\theta = \frac{1}{N_x/2}\arcsin\left(\frac{R}{d}\right)\ [\mathrm{rad}] \approx \frac{R}{d\,(N_x/2)}\ [\mathrm{rad}]. \tag{29}$$

For $R = 1$, $d = 100$, $N_x = 129$: $\delta\theta \approx 0.^\circ0089$, covering $\pm 0.^\circ57$ (total $1.^\circ14$), which exactly spans the sphere.

Clump-aware lab-frame conversion: At the moment a peel-off is computed, `photon%xfreq` is in the host clump's rest frame (non-overlap path) or in the global lab frame (overlap path; see §15.3). The peel-off output must be in the lab (observer) frame, hence the routines compute

$$x_{\mathrm{ref}} = x_{\mathrm{photon}} + u_1, \qquad u_1 = \mathbf{3}_{\mathrm{cell}} \cdot \hat{\mathbf{k}}_{\mathrm{obs}} / v_{\mathrm{th}}. \tag{30}$$

For the standard Cartesian path $\mathbf{3}_{\rm cell}$ comes from `grid%vfx/y/z`; in clump mode `grid%vfx/y/z` are zero everywhere (the inter-clump medium is vacuum and the bulk velocities live on the clumps), so the peel-off routines branch on `par%use_clump_medium .and. photon%icell_clump > 0` and use the host clump's velocity instead:

$$u_1 = \texttt{cl\_vx(icl)}\,k_x + \texttt{cl\_vy(icl)}\,k_y + \texttt{cl\_vz(icl)}\,k_z, \tag{31}$$

where `icl` = `photon%icell_clump`.

This branch applies both to photons that scattered inside a clump (the usual case) and to photons *born* inside a clump. Since `generate_photon_car` now sets `icell_clump` and shifts `xfreq` into the clump rest frame at birth whenever the birth point falls inside a clump (non-overlap path), `peeling_direct_outside` picks up the same branch. Before this fix, `icell_clump` was always 0 at birth, so the direct peel-off spectrum was binned in the clump rest frame instead of the lab frame, producing a systematically shifted line profile.

Without this branch the peel-off spectra would remain in the clump frame while `grid%Jout` (the angular average) is in the lab frame, producing a systematically narrower peel-off line profile whenever the clump bulk velocity field is non-trivial (e.g. rotating-halo or Hubble-flow runs).

FITS WCS for the 3D peel-off cube: The peel-off cube is allocated as `obs%scatt(par%nxfreq, par%nxim, par%nyim)`, so the FITS file has `NAXIS1` = `nxfreq` (wavelength), `NAXIS2` = `nxim` (RA), `NAXIS3` = `nyim` (DEC). The header carries a three-axis WCS:

Axis	CTYPE	CUNIT	step
NAXIS1	WAVE	Angstrom	CD1_1 $= -\delta\lambda$
NAXIS2	RA--TAN	deg	CD2_2 $= \delta\theta_x$
NAXIS3	DEC-TAN	deg	CD3_3 $= \delta\theta_y$

`CD1_1` is negative because $x_{\rm freq}$ increases with the pixel index while wavelength $\lambda(x) = (1 - v_{\rm th}\,x/c)\,\lambda_0$ decreases with x. `CRPIX1` = 1, `CRVAL1` = `grid%wavelength(1)` (largest wavelength, at pixel 1). The scalar header keywords `XFREQ1`, `XFREQ2`, `DXFREQ`, and `DWAVE` are also retained for backward compatibility with existing analysis scripts that compute the spectral axis from those instead of from the WCS.

8. FITS Output of Clump Information

When `par%save_clump_info = .true.`, the subroutine `write_clumps_fits` (in `clump_mod.f90`) writes a binary-table FITS file `<base>_clumps.fits.gz` containing:

BinTable columns: `X`, `Y`, `Z` (clump centers, code units) and `VX`, `VY`, `VZ` (bulk velocities, km/s) are always written. The internal arrays `cl_vx/y/z` are stored dimensionlessly as $\mathbf{3}/v_{\rm th}$; `write_clumps_fits` multiplies each component by `cl_vtherm` before writing so that the file always carries velocities in km/s.

In addition the clump physical columns `R_CLUMP`, `RHOKAP`, and `TEMP` are written when they carry useful spatial variation. For each one independently the relative spread $(\max - \min)/|\text{mean}|$ is compared against 10^{-3}; if the column is below that threshold (i.e. effectively constant) it is dropped and only the corresponding header keyword (`CL_RAD`, `RHOKAP`, `TEMP_CL`) is kept. The diagnostic line printed by `write_clumps_fits` reports the decision for each column, e.g.

```
Clumps: clump columns saved (R_CLUMP/RHOKAP/TEMP) =  T F F
```

When `read_clumps_fits` consumes the file and a column is absent, the corresponding `cl_*` array is filled uniformly with the header-keyword value. This makes the read path equally able to handle: (i) pre-Phase-6 files which never had these columns; (ii) new files where a column was dropped because it was constant; and (iii) externally produced files where the column carries genuine clump-to-clump variation.

For the `RHOKAP` slot the read path additionally accepts `DENSITY` and `DENS` as alternative column names: the priority order is `RHOKAP` → `DENSITY` → `DENS`, with the header keyword `RHOKAP` as the final fallback. This lets clump files generated by external pipelines that label the clump opacity proxy differently be consumed without renaming columns. The semantic interpretation depends on which column was actually used:

- `RHOKAP` (and the header-keyword fallback) is taken as line-centre opacity per code unit, identical to the convention written by `write_clumps_info`, and stored directly into κ_i.
- `DENSITY` / `DENS` are taken as the neutral-hydrogen (or analog-ion) number density $n_{HI,i}$ [cm^{-3}] and converted to opacity via

$$\kappa_i = n_{HI,i} \cdot \frac{\texttt{line\%cross0}}{\Delta\nu_{D,i}} \cdot \texttt{par\%distance2cm}, \tag{32}$$

  using the clump Doppler frequency $\Delta\nu_{D,i}$ derived from T_i and the line-centre cross section `line%cross0`. When `par%distance_unit` (and therefore `par%distance2cm`) is not specified, the conversion is undefined; the read path emits a warning and falls back to treating the values as κ_i (the legacy interpretation).

Header keywords (in the BinTable HDU):

Keyword	Value	Description
N_CLUMPS	N	number of clumps (realized)
SPHERE_R	R	outer sphere radius (code units)
CL_RAD	r_c	clump radius (code units)
F_VOL	$f_{3\mathrm{ol}}$	volume filling factor (realized)
F_COV	f_{co3}	covering factor (realized)
TAU0	τ_0	line-center τ per clump
SIGMA_V	σ_v	bulk velocity dispersion (km/s)
TEMP_CL	T_c	clump temperature used (K)
RHOKAP	κ_c	opacity per code unit
CL_DFREQ	$\Delta\nu_D^{cl}$	Doppler frequency (Hz)
VTHERM	v_{th}	thermal velocity (km/s)
VOIGT_A	a_V	Voigt damping parameter
RMAX	R	par%rmax (input)
IN_FCOV	par%clump_f_cov	input covering factor
IN_FVOL	par%clump_f_vol	input filling factor
IN_NCL	par%clump_N_clumps	input N
IN_NHI	par%clump_NHI	clump N_{HI} (cm^{-2}, clump center to surface)
IN_NH	par%clump_nH	input n_H (cm^{-3})
IN_TEMP	par%clump_temperature	input T_c (K)
DISTUNIT	par%distance_unit	distance unit (kpc/pc/...)
DIST_CM	par%distance2cm	1 code unit in cm

DISTUNIT and DIST_CM carry the distance unit and the code-unit-to-cm conversion factor at the time the file was written. When read_clumps_fits loads a file, it uses these keywords as fallback values for par%distance_unit / par%distance2cm: if the user supplied a distance unit in the input namelist (par%distance_unit /= '' *and* par%distance2cm > 1), the namelist value is kept; otherwise the file keywords are adopted. This matters in particular for the DENSITY/DENS-to-opacity conversion in Eq. 32: without a known distance scale, the read path falls back to interpreting the column as κ_i (with a warning). External pipelines that write LaRT-compatible HDF5 must store string attributes as *fixed-length* ASCII (e.g. h5py's string_dtype(encoding='ascii', length=N)); variable-length strings are detected and skipped, since the HDF5 Fortran binding cannot read them through the same call path used here.

Note: column data are written first (which creates the BinTable HDU); header keywords are then written into that same HDU. FITS keyword names must not exceed 8 characters.

9. Input Parameters Summary

Parameter	Default	Description
par%grid_type	'car'	'clump' selects clump mode (v2.10)
par%use_clump_medium	.false.	deprecated alias: maps to grid_type='clump'
par%rmax	—	outer sphere radius (code units)
par%clump_radius	—	clump radius r_c (code units)
par%clump_f_cov	−1	covering factor (input; specify one of f_cov/f_vol/N_clumps)
par%clump_f_vol	−1	volume filling factor (input)
par%clump_N_clumps	−1	number of clumps (input, direct)
par%clump_tau0	−1	line-center τ of one clump (center to surface)
par%clump_NHI	−1	clump $N_{\rm HI}$ (cm^{-2}, clump center to surface; no distance_unit needed)
par%clump_nH	−1	clump n_H (cm^{-3}; requires distance_unit)
par%clump_temperature	−1	clump T (K); $< 0 \Rightarrow$ par%temperature
par%clump_sigma_v	0	random velocity σ per component (km/s)
par%velocity_type	''	systematic flow type (see §3.4)
par%Vexp	0	expansion speed for hubble/constant_radial/power_law/linear_decelerate (km/s)
par%velocity_alpha	1	power-law exponent for power_law velocity type
par%Vx, Vy, Vz	0	bulk velocity for parallel_velocity (km/s)
par%Vpeak	0	peak speed for ssh (km/s)
par%rpeak	0	peak radius for ssh (code units)
par%DeltaV	0	outer velocity increment for ssh (km/s)
par%Vrot	0	rotation speed for rotating models (km/s)
par%rinner	0	solid-body radius for rotating_galaxy_halo
par%save_clump_info	.false.	write clump FITS file
par%clump_input_file	''	if non-empty, load the clump population from this FITS file (see §13.) instead of generating internally

Example input (v2.10 OOP interface):

```
&parameters
 par%grid_type        = 'clump'
 par%rmax             = 1.0
 par%clump_radius     = 0.01
 par%clump_f_cov      = 5.0
 par%clump_tau0       = 1.0e3
 par%temperature      = 1.0e4
 par%clump_sigma_v    = 0.0
 par%no_photons       = 1e6
 par%spectral_type    = 'voigt'
```

```
  par%nxfreq           = 301
  par%velocity_min     = -100.0
  par%velocity_max     =  100.0
  par%nx = 11, par%ny = 11, par%nz = 11
  par%save_clump_info  = .true.
  par%out_file         = 'clump_tau3_fcov5.fits.gz'
 /
```

Backward-compatible input (v2.00 style, accepted by v2.10):

```
 &parameters
  par%use_clump_medium = .true.    ! mapped to par%grid_type = 'clump'
  par%rmax             = 1.0
  par%clump_radius     = 0.01
  par%clump_f_cov      = 5.0
  par%clump_tau0       = 1.0e3
  par%temperature      = 1.0e4
  par%no_photons       = 1e6
  par%spectral_type    = 'voigt'
  par%nxfreq           = 301
  par%velocity_min     = -100.0
  par%velocity_max     =  100.0
  par%nx = 11, par%ny = 11, par%nz = 11
  par%save_clump_info  = .true.
  par%out_file         = 'clump_tau3_fcov5.fits.gz'
 /
```

10. Memory Layout

Clump positions, velocities, and CSR arrays are allocated as MPI-3 shared-memory windows via `create_shared_mem` from `memory_mod`, giving one copy per compute node rather than per MPI rank. On systems with multiple ranks per node this significantly reduces memory usage for large clump populations.

Cleanup is performed by `destroy_clumps` which calls `destroy_shared_mem_all` (collective `MPI_WIN_FREE` across all ranks) followed by explicit `nullify` of all pointers. The individual `destroy_mem` is not used because `MPI_WIN_SHARED_QUERY` returns the rank-0 base address, which differs from the local pointer on non-root ranks.

11. Validation and Examples

The directory `examples/clump_sphere/` contains a suite of six reference runs covering the main parameter space. All use $R = 1$, $r_c = 0.01$, 10^6 photons, and `par%grid_type = 'clump'`:

Input file	τ_0	Notes
`clump_tau3_fcov5.in`	10^3	$f_{\rm co3} = 5$, static (reference)
`clump_tau4_fcov5.in`	10^4	$f_{\rm co3} = 5$, static
`clump_tau3_fcov20.in`	10^3	$f_{\rm co3} = 20$, dense covering
`clump_tau3_fcov5_hubble.in`	10^3	$f_{\rm co3} = 5$, Hubble $V_{\rm exp} = 200$ km/s
`clump_tau3_fcov5_sigv.in`	10^3	$f_{\rm co3} = 5$, $\sigma_v = 50$ km/s
`clump_tau3_fcov5_peel.in`	10^3	$f_{\rm co3} = 5$, peel-off 129×129

Run with `bash run.sh` (uses 4 MPI ranks by default) or individually:

```
cd LaRT_v2.10/examples/clump_sphere
mpirun -np 4 ../../LaRT.x clump_tau3_fcov5.in
```

Python post-processing:

```
python plot_clump.py          # all runs
python plot_clump.py clump_tau3_fcov5   # single run
```

For the reference case ($f_{\rm co3} = 5$, $\tau_0 = 10^3$):

- Realized $f_{\rm co3} \approx 5.00$, $f_{\rm 3ol} \approx 0.067$;
- RSA acceptance rate $\sim 74\%$, $N = 66667$ clumps placed;
- Mean scattering count ≈ 6400 per photon;
- Wall time ≈ 3.75 min (10 ranks, 10^5 photons).

The escaped spectrum $J_{\rm out}(x)$ shows the characteristic double-peak expected for a clumpy Lyman-α source. The peel-off output (`_obs.fits.gz`) contains `Scattered` (HDU 0) and `Direct` (HDU 1) spectral image cubes of shape $(301, 129, 129)$.

Sight-line tau in clump mode: A clump-aware sight-line traversal is now implemented in `sightline_tau_clump.f90`; the type-bound dispatch `clump_make_sightline_tau` (v2.10) and the procedure-pointer assignment in `setup_procedure` (v2.00) route to it whenever `par%save_sightline_tau = .true.`. The standalone driver `make_sightline_tau.x` consumes the same input file and writes `<base>_tau.fits.gz` (`TAU_gas`, `N_gas`, optionally `TAU_dust`). See `examples/sightline_tau/clump_sightline.in` for a complete reference run.

Peel-off vs. Jmu validation: The notebook `verify_peel_jmu.ipynb` (in `examples/clump_sphere/`) checks that the integrated peel-off Stokes-I spectrum matches the sky-mean of `Jmu` (the angle-binned escape spectrum). Cells covering $f_{\rm co3} = 1$, 2, and 5 confirm that the measured `Scatt/Jmu` reproduces the analytic single-scattering escape fraction $1 - \exp(-f_{\rm co3})$ to 1 % across four observer β angles. The $f_{\rm co3} = 5$ case in particular validates the new `tau_huge_clump = 745.2` cap added to the peel-off raytrace (Sect. "Peel-off"): even at $\tau_{\rm max} \sim 6.7 \times 10^3$ the cap preserves the answer because every capped contribution has $\exp(-\tau) \approx 0$.

12. Spatially-Varying Clump Properties

The default clump population is uniform in radius, internal density, and spatial number density. Three optional radial-profile inputs allow each of these to vary as a function of distance from the box center. When all three profile flags are set to `'constant'` (the default) the behavior is identical to the uniform implementation.

12.1 Input parameters

```
par%clump_radius_profile  = 'constant' | 'powerlaw' (or 'power_law')
                          | 'gaussian' | 'exponential' | 'file'
par%clump_density_profile = (same options ; controls n_H(r) inside clump)
par%clump_number_profile  = (same options ; controls spatial n_cl(r))
par%clump_radius_alpha,   par%clump_radius_r0    ! r0 <= 0 -> rmax
par%clump_density_alpha,  par%clump_density_r0   ! r0 <= 0 -> rmax
par%clump_number_alpha,   par%clump_number_r0    ! r0 <= 0 -> rmax
par%clump_profile_file    = ''   ! 5-column tabulated profile
par%cone_opening          = 0.0  ! bicone half-opening angle [deg]; 0 = sphere
```

When `cone_opening` > 0, clumps are placed only inside the biconical region ($|\cos\theta| \geq \cos\alpha_{\rm cone}$, both hemispheres along the z-axis). Angular positions are sampled directly from $\cos\theta \in [\cos\alpha_{\rm cone}, 1]$ (no rejection needed). When `clump_fully_inside = .true.`, clumps whose sphere would protrude outside the cone boundary are additionally rejected: the angular subtension $\delta = \arcsin(r_c/r)$ of the clump is used to verify that $\cos(\theta_c + \delta) \geq \cos\alpha_{\rm cone}$.

When `r0` ≤ 0 (the default), it is automatically set to `par%rmax` (i.e. `sphere_R`), so that $f(r_{\rm max}) = 1$ for the power-law, Gaussian, and exponential profiles.

Built-in shapes (same convention on each axis; f multiplies the user-supplied peak value):

$$\begin{aligned}
\text{constant} &: f(r) = 1, \\
\text{powerlaw} &: f(r) = \left(\max(r, r_{\rm fl})/\max(r_0, r_{\rm fl})\right)^{-\alpha}, \quad r_{\rm fl} = 0.05\, r_0, \\
\text{gaussian} &: f(r) = \exp\left[-(r/r_0)^2\right], \\
\text{exponential} &: f(r) = \exp(-r/r_0).
\end{aligned}$$

For `'file'`, the named ASCII file should contain columns `r`, `r_cl_factor`, `n_H_factor`, `T[K]`, `n_cl_shape`, one row per radial sample point, and the relevant column is interpolated linearly.

12.2 Covering factor under non-uniformity

The covering factor with non-uniform profiles is defined as the radial line-of-sight integral

$$f_{\rm co3} = \int_0^R n_c(r)\, \pi\, r_c(r)^2\, dr, \tag{33}$$

which reduces to the uniform formula $\frac{3}{4} N (r_c/R)^2$ when all three profiles are constant. See `covering_factor_definitions.tex` for the derivation and alternatives.

12.3 Implementation outline

- Clump physical quantities (`cl_radius`, `cl_rhokap`, `cl_voigt_a`, `cl_Dfreq`, `cl_vtherm`, `cl_temperature`) are MPI shared-memory arrays of dimension `N_clumps`; reference scalars (`cl_*_ref`) carry the peak values.
- Clump positions are sampled by inverse CDF on $n_c(r)4\pi r^2$, with isotropic angles. The RSA 27-neighbor overlap test compares $(r_c^i + r_c^j)^2$ so non-uniform clump radii are handled correctly.
- `N_clumps` is derived in two ways: closed-form for the uniform case, numerical quadrature with the LOS $f_{\rm co3}$ definition otherwise. Any one of `clump_N_clumps`, `clump_f_vol`, `clump_f_cov` can be specified; the other two are obtained from quadrature.
- Clump opacity in the temperature-varying case picks up the Doppler-frequency rescaling

$$\rho\kappa(\mathbf{r}) = \rho\kappa_{\rm peak}\,\frac{n_H}{n_{H,\rm peak}}\,\frac{\Delta\nu_D^{\rm ref}}{\Delta\nu_D(\mathbf{r})}, \tag{34}$$

  so that the local cross-section $\sigma_0/\Delta\nu_D$ is consistent with the local thermal width.
- The clump FITS table now includes clump `R_CLUMP`, `RHOKAP`, `TEMP` columns in addition to positions and velocities. Stdout prints the realized $f_{\rm 3ol}$, $f_{\rm co3}$, r_c range, mean $\tau_{\rm per\,clump}$, and integrated HI mass.

12.4 Example inputs

```
! Gaussian-concentrated clump cloud (constant clump radius)
par%clump_number_profile = 'gaussian'
par%clump_number_r0      = 0.4
par%clump_f_cov          = 5.0

! Clump radius growing linearly with r (constant n_cl).
! Specifying clump_f_cov lets the profile-aware quadrature back-solve
! the spatial normalization; N_clumps is reported from the integral.
par%clump_radius_profile = 'powerlaw'
par%clump_radius_alpha   = -1.0
par%clump_radius_r0      = 0.5
par%clump_f_cov          = 1.0
```

For full working setups see, in `examples/clump_sphere/`:

- `clump_powerlaw_density.in`
- `clump_radius_powerlaw.in`

13. Standalone Clump Generator (`make_clumps.x`)

The clump-generation pathway used inside LaRT is also exposed as a self-contained standalone executable so that a clump population can be produced once and reused across many transport runs (or shared with external analyses). The

driver source is `make_clumps.f90`; build with

```
make make_clumps        # builds make_clumps.x
```

Unlike previous releases, the driver no longer links against MPI or the LaRT physics stack: the link footprint is limited to `define.o`, `fitsio_mod.o`, `hdf5io_mod.o`, `iofile_mod.o`, and `voigt_mod.o`. The RNG is an inlined xorshift64* generator (deterministic, MPI-free) so the FITS/HDF5 table is no longer byte-identical to the one produced by a regular `LaRT.x` run with `par%save_clump_info = .true.`; it is, however, schema-identical and statistically equivalent (matched $f_{\rm co3}$, $f_{\rm 3ol}$, τ_0, line, and velocity field).

Usage. The driver no longer reads a namelist. All parameters are supplied as command-line flags (one flag per `par%clump_*` field plus the relevant global parameters), mirroring the Python port:

```
# uniform sphere with covering factor 5, Lyman-alpha tau_0 = 1e3
./make_clumps.x --rmax 1.0 --clump_radius 1e-3 \
                --clump_f_cov 5 --clump_tau0 1e3 \
                --temperature 1e4 --line_id ly_alpha \
                --seed 12345 -o run_clumps.fits.gz

# biconical Hubble outflow, 45 deg half-opening
./make_clumps.x --rmax 1.0 --clump_radius 0.02 \
                --clump_f_cov 2 --cone_opening 45 \
                --clump_tau0 1e3 --velocity_type hubble --Vexp 200 \
                -o bicone_clumps.h5
```

Run `./make_clumps.x --help` for the full flag list. The output filename's extension selects the container: `.fits` / `.fits.gz` produce a FITS binary table, while `.h5` / `.hdf5` produce an HDF5 table; the on-disk schema (header keywords + columns) is identical in both cases and matches §8., including the column-skipping rule for `R_CLUMP` / `RHOKAP` / `TEMP`.

Loading the file from LaRT. Pass the path through `par%clump_input_file`:

```
&parameters
 par%grid_type        = 'clump'
 par%rmax             = 1.0
 par%clump_radius     = 0.01
 par%clump_tau0       = 1.0e3
 par%temperature      = 1.0e4
 par%no_photons       = 1e6
 par%spectral_type    = 'voigt'
 par%nxfreq           = 301
 par%nx = 11, par%ny = 11, par%nz = 11
 par%out_file         = 'clump_run.fits.gz'

 ! Reuse a previously generated population
```

```
 par%clump_input_file = 'clumps_input_clumps.fits.gz'
/
```

When `par%clump_input_file` is non-empty, `init_clumps` skips the profile detection and RSA-generation path entirely. The population is read from the FITS file and distributed across MPI ranks using the standard host-comm / `SAME_HRANK_COMM` pattern; the CSR acceleration grid is then rebuilt deterministically on every node. The internal `generate_clumps` routine is left untouched so that runs without `clump_input_file` reproduce previous results byte-for-byte.

Supported features. The Fortran driver supports the full clump configuration space:

- uniform and radial-profile (`powerlaw`, `gaussian`, `exponential`) placement;
- biconical geometry via `--cone_opening` (with `--clump_fully_inside` enforcing the cone-edge angular-subtension test);
- all opacity-input modes (`--clump_tau0`, `--clump_NHI`, `--clump_nH`, `--taumax`, `--N_HImax`);
- the full line catalogue used by LaRT (`--line_id`);
- the same systematic-velocity families as the LaRT engine (`hubble`, `constant_radial`, `power_law`, `linear_decelerate`, `parallel_velocity`, `ssh`, `rotating_solid_body`, `rotating_galaxy_halo`) on top of the Gaussian `--clump_sigma_v` random component.

Python equivalent (`make_clumps.py`). A pure-Python port of the same generator lives at `python/make_clumps.py`; it exposes an essentially identical flag set and writes the same FITS/HDF5 schema described in §8. (`EXTNAME='Clumps'`, columns `X/Y/Z/VX/VY/VZ` plus the optional `R_CLUMP/RHOKAP/TEMP` survivors, full header keyword set including `CONE_OP` and `LINE_ID`). Either driver's output is directly consumable by LaRT via `par%clump_input_file`. The two drivers use independent RNG implementations, so for a given seed they produce statistically equivalent but not bit-identical populations; pick whichever fits the host environment (no Fortran toolchain → Python; no Python → Fortran).

Reproducibility. Both drivers are deterministic in `--seed`: re-running with the same flag set and the same seed reproduces the table bit-for-bit. Cross-driver (Fortran vs. Python) and cross-version (this release vs. the legacy LaRT-internal generator) reproducibility is only statistical, not bitwise.

14. Standalone Sight-line Tau Calculator (`make_sightline_tau.x`)

A second standalone driver, `make_sightline_tau.f90`, builds the sight-line maps for each observer without running the Monte Carlo photon loop. The output FITS file matches the layout produced by an ordinary LaRT run with `par%save_sightline_tau = .true.` (see the LaRT user manual, §Observers and Peel-off):

HDU	Content
TAU_gas	gas optical depth $\tau(\nu, x, y)$
N_gas	HI column density $N_{\rm HI}(x, y)$
TAU_dust	dust optical depth $\tau_{\rm dust}(x, y)$ (only when par%DGR> 0)

Build and run.

```
make sightline_tau                              # builds make_sightline_tau.x
mpirun -np N ./make_sightline_tau.x sl.in       # writes <base>_tau.fits.gz
```

Clump-mode wiring. The default sight-line path inside LaRT (sightline_tau_rect) uses Cartesian box-face geometry and the raytrace_to_edge_car dispatch. In clump mode grid%rhokap= 0 everywhere; opacity lives in cl_rhokap(:) and is reached only by the CSR DDA traversal in raytrace_clump.f90. A new module, sightline_tau_clump.f90, mirrors sightline_tau_amr.f90 but for clump mode: it intersects the bounding sphere of radius sphere_R (instead of the AABB), initializes photon%icell_clump = 0, and dispatches to the correct raytrace variant.

In v2.10 the type-bound clump_make_sightline_tau calls this module; in v2.00 the procedure pointer make_sightline_tau is set to make_sightline_tau_clump inside the use_clump_medium branch of setup_procedure.

When has_overlap = .false. (default, non-overlapping clumps), make_sightline_tau_clump calls raytrace_to_edge_tau_gas_clump and raytrace_to_edge_column_clump directly. When has_overlap = .true., the overlap-aware variants raytrace_to_edge_tau_gas_clump_overlap and raytrace_to_edge_column_clump_overlap are used instead, ensuring that the sightline $\tau(\nu)$ and $N_{\rm HI}$ maps correctly account for overlapping opacities. In v2.10 an inline if (has_overlap) check selects the variant directly. In v2.00 sightline_tau_clump.f90 calls the global procedure pointers raytrace_to_edge_tau_gas and raytrace_to_edge_column; these are upgraded to the _overlap variants by setup_clump_overlap() (called from main.f90 after grid_create_clump() sets has_overlap).

Example inputs.

- examples/clump_sphere/clump_sightline_test.in
- examples/sphere/sphere_sightline_test.in

15. Overlap-Aware Raytrace

15.1 Motivation

The RSA placement algorithm guarantees that internally generated clumps never overlap. However, clump populations loaded from an external FITS file via par%clump_input_file carry no such guarantee. The original raytrace assumed non-overlap and produced incorrect results when two clumps shared volume:

- At clump-A exit, `photon%icell_clump` was reset to 0 (vacuum) without checking whether the photon was simultaneously inside clump B.
- The opacity in the overlap region was under-counted.
- Frequency shifts at clump boundaries were mis-applied.

15.2 Physical Interpretation

Two overlapping clumps with distinct bulk velocities $\mathbf{3}_1$ and $\mathbf{3}_2$ are treated as two independent gas components coexisting at the same spatial location – the standard multi-component RT picture. The total line opacity at global-frame frequency x along direction $\hat{k}$ is

$$\kappa_{\rm total}(x) = \sum_i \rho\kappa_i H\big(x - u_{{\rm los},i},\, a_i\big), \qquad u_{{\rm los},i} = \frac{\mathbf{3}_i \cdot \hat{k}}{\Delta\nu_{D,i}}. \tag{35}$$

When a scattering event falls in an overlap region the interacting atom belongs to one specific clump, chosen by opacity-weighted sampling:

$$P(\text{scatter in clump } i) = \kappa_i(x)/\kappa_{\rm total}(x). \tag{36}$$

15.3 Frequency Convention

The two raytrace paths use different conventions for `photon%xfreq`.

Non-overlap path (`has_overlap = .false.`): `photon%xfreq` tracks the *current rest frame* of the photon:

- In inter-clump vacuum: global (lab) frame.
- Inside a clump: that clump's rest frame.

Frame transitions occur at every clump boundary crossing:

$$\text{entry:}\quad \texttt{photon\%xfreq} \mathrel{-}= u_{\rm los} \qquad (\text{global} \to \text{clump frame}), \tag{37}$$

$$\text{exit:}\quad \texttt{photon\%xfreq} \mathrel{+}= u_{\rm los} \qquad (\text{clump frame} \to \text{global}). \tag{38}$$

The Voigt opacity evaluation uses `photon%xfreq` directly because it is already in the clump frame. Photons *born inside a clump* are handled symmetrically: `generate_photon` calls `active_set_at_point` at birth; if the birth point falls inside a clump, `photon%icell_clump` is set to the host clump index and `photon%xfreq` is shifted into the clump rest frame immediately, before the first peel-off call.

Unit convention for clump temperature. `photon%xfreq` is carried globally in units of $\Delta\nu_{D,\rm ref} \equiv$ `cl_Dfreq_ref`, the reference Doppler frequency. Clump arrays use their own local units: `cl_voigt_a`$(i) = \Gamma/(4\pi\,\Delta\nu_{D,i})$, and `cl_vx/y/z`$(i) = v_i/v_{\rm th}^{(i)}$ with $v_{\rm th}^{(i)} = \lambda_0\,\Delta\nu_{D,i}$. When the clump temperature varies along the radius ($\Delta\nu_{D,i} \neq \Delta\nu_{D,\rm ref}$) the Voigt evaluation and the bulk-velocity Doppler shift each require a rescale to remain self-consistent:

$$H\big(x, a_i\big)\big|_{x_{\rm loc}} \quad \text{with} \quad x_{\rm loc} = \texttt{photon\%xfreq} \cdot \frac{\Delta\nu_{D,\rm ref}}{\Delta\nu_{D,i}}, \tag{39}$$

$$u_{\rm los} = \big(\texttt{cl\_vx}(i)\,k_x + \texttt{cl\_vy}(i)\,k_y + \texttt{cl\_vz}(i)\,k_z\big) \cdot \frac{\Delta\nu_{D,i}}{\Delta\nu_{D,\rm ref}}. \tag{40}$$

Both expressions reduce to the previous (unrescaled) forms when $\Delta\nu_{D,i} = \Delta\nu_{D,\mathrm{ref}}$, so uniform-$T$ runs are unaffected. The two helpers

```
voigt_clump(xfreq, icl)        ! returns H(x_loc, cl_voigt_a(icl))
ulos_clump(icl, kx, ky, kz)    ! returns u_los (REF Doppler units)
```

in `clump_mod` apply the rescaling once and are used wherever a Voigt opacity or a clump-boundary frequency shift is needed: `raytrace_clump` (non-overlap and overlap paths), `peelingoff_rect` (clump-frame peel-off conversion), `generate_photon` (birth-inside-clump shift), and the `scatter_resonance_clump_*` overlap wrappers.

Resonance scattering: `do_resonance_*_clump` variants. The line-type-dependent dispatch routine `do_resonance` samples the absorbing-atom velocity `uz`, the atom-frame frequency `xfreq_atom`, and the multiplet branch (where applicable). For Cartesian and AMR grids it reads `grid%voigt_a` and `grid%Dfreq` per cell. In the clump medium these grid arrays are filled uniformly with the reference values and would therefore ignore clump T entirely. A separate module `line_clump_mod` provides clump-aware variants `do_resonance{1,2,4,5,6,HD}_clump` that read `cl_voigt_a(icl)` and `cl_Dfreq(icl)` directly, rescale `photon%xfreq` to clump-local Doppler-width units for sampling, and convert the returned `uz` and `xfreq_atom` back to REF units before returning. `setup_procedure` wires these variants per `line%line_type` when `par%use_clump_medium = .true.`. The caller (`scatter_resonance_{nostokes,stokes}`) additionally scales the perpendicular thermal velocities u_x, u_y by $\Delta\nu_{D,i}/\Delta\nu_{D,\mathrm{ref}}$ and computes the recoil shift using `cl_Dfreq(icl)` so that the entire scattering kernel is consistent with the local clump temperature.

Overlap path (`has_overlap = .true.`): `photon%xfreq` is *always* in the global (lab) frame. The clump Doppler shift is applied on the fly inside `sum_kap_active` and `collect_ray_events_overlap`:

$$x_{\mathrm{cl}} = x_{\mathrm{freq},g} - u_{\mathrm{los},i}, \tag{41}$$

evaluated independently for each active clump i. This is necessary because the photon may be simultaneously inside several clumps with different bulk velocities, so there is no single "current clump frame".
The Doppler shift at scatter time is handled by the wrapper routines `scatter_resonance_clump_nostokes` / `scatter_resonance_clump_stokes` (selected by `par%use_stokes`).

Consequence for peel-off: The lab-frame conversion $x_{\mathrm{ref}} = \texttt{photon\%xfreq} + u_1$ requires $u_1 = \mathbf{3}_{\mathrm{cl}} \cdot \hat{\mathbf{k}}_{\mathrm{obs}}/v_{\mathrm{th}}$ when `photon%icell_clump` > 0 (non-overlap; xfreq in clump frame), and $u_1 = 0$ when `photon%icell_clump` $= 0$ (vacuum inter-clump region, or the overlap path where xfreq is always in the lab frame).

Design note: Unifying both paths to the global-frame convention is feasible in principle—the non-overlap path would compute $x_{\mathrm{cl}} = x_{\mathrm{freq},g} - u_{\mathrm{los}}$ at each opacity evaluation instead of at entry/exit boundaries. This is currently not recommended: the non-overlap path is well-tested, the clump frame convention is natural when only one clump is active at a time, and the change would require rewriting all seven `raytrace_*_clump` functions. It could be revisited if future requirements make a unified global-frame convention desirable.

15.4 Overlap Detection

After the CSR grid is built, `check_has_overlap()` scans all clump pairs using the existing CSR 27-neighbor structure — $\mathcal{O}(N \times k_{\rm nb})$ time. The result is stored in the module variable `has_overlap` (default `.false.`). For RSA-generated populations `has_overlap` is never set to `.true.`: zero runtime overhead for the common case.

15.5 Event-Based Segment Raytrace

`collect_ray_events_overlap` performs a full DDA scan – no early exit – and collects every clump intersection as a sorted list of $(t, i_{\rm cl}, \pm 1)$ events (entry or exit). Clumps that already contain the ray origin emit only an EXIT event. `active_set_at_point` queries the CSR 27-neighbor cells to find all clumps containing the starting point, initializing the active set.
`raytrace_to_tau_clump_overlap` then walks the sorted event list:

1. For the current interval $[t_{\rm cur}, t_{\rm next}]$, compute $\kappa_{\rm total}$ from the active set.
2. If $\kappa_{\rm total} \cdot \Delta t \geq \tau_{\rm rem}$, advance to the scatter point; choose the owner clump by opacity-weighted sampling via `sample_owner_clump`; store the owner in `photon%icell_clump`; return.
3. Otherwise subtract $\kappa_{\rm total} \cdot \Delta t$ from $\tau_{\rm rem}$ and update the active set at the event boundary.

The read-only variants (`_edge_*` and `_dist_*`) use the same event list but simply accumulate τ or $N_{\rm HI}$ without a scatter decision.

15.6 Enabling Overlap for Internally Generated Clumps

A new parameter allows the RSA overlap rejection to be bypassed:

```
par%clump_allow_overlap = .true.   ! default: .false.
```

When `.true.`, clumps are placed at uniformly random positions without checking for overlap. After placement, `check_has_overlap()` is called automatically; if any overlapping pair is found the overlap-aware raytrace is engaged. A typical test configuration using large clumps and a low covering factor to guarantee overlaps quickly:

```
par%clump_allow_overlap = .true.
par%clump_radius        = 0.3     ! large clumps
par%clump_f_cov         = 2.0     ! few clumps -> fast placement
par%no_photons          = 1e5     ! fast MC run
```

Reference example inputs are in `examples/clump_sphere/`: `clump_overlap_mono.in`, `clump_overlap_sigv.in`, `clump_overlap_peel.in`.

15.7 Performance Notes

Scenario	Overhead
RSA-generated, non-overlap	None – original code path
File-loaded, no overlap	`check_has_overlap()` once at init; original raytrace
Overlap present	Event-based raytrace; $\mathcal{O}(n_{\text{acti3e}})$ per segment